การจัดวาง Layout และการจัดการ Event โดยใช้ Starter Template ที่เป็นโครงสร้างแบบมืออาชีพ (Functional Component)

---

# Function 7.1 - The Personal Link-in-Bio App

---

### 1. วัตถุประสงค์

1. ฝึกการใช้งาน Core Componentsพื้นฐาน ( `View` , `Text` , `Image` , `Pressable` )
2. เข้าใจการใช้ `StyleSheet` ในการจัด Layout ด้วย `Flexbox`
3. ฝึกการจัดการ State และ Event ในการตอบสนองต่อการสัมผัส

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

### 2. โจทย์โดยละเอียด

พัฒนาแอปพลิเคชันหน้าเดียว (Single Page) เพื่อรวบรวมลิงก์ส่วนตัว โดยมีเงื่อนไขดังนี้:

1. **Profile Section**: แสดงรูปภาพโปรไฟล์ (ใช้ URL จากเน็ตได้) โดยรูปต้องเป็น **วงกลมสมบูรณ์**
2. **Dynamic Style**: ทุกครั้งที่กดปุ่มใดปุ่มหนึ่ง ขอบของรูปโปรไฟล์ต้อง **เปลี่ยนสีแบบสุ่ม (Random Color)**
3. **Buttons Section**: สร้างปุ่มอย่างน้อย 3 ปุ่ม โดยใช้ `Pressable`
4. **Interaction**: เมื่อกดปุ่ม ให้แสดง `Alert` ระบุชื่อแพลตฟอร์มที่กด

---

### 3. ขั้นตอนการทำ (Instructions)

#### Step 1: จัดการ Layout

ไปที่ `styles.container` และใช้ `justifyContent` ร่วมกับ `alignItems` เพื่อให้ทุกอย่างอยู่กลางหน้าจอ

#### Step 2: จัดการรูปภาพ (The Circular Image)

ในมือถือ การทำรูปวงกลมมีหลักการคือ `borderRadius` ต้องมีค่าเป็น **ครึ่งหนึ่งของความกว้าง/สูง** จงคำนวณและใส่ค่าที่ถูกต้องใน `styles.profileImage`

#### Step 3: ตรรกะการสุ่มสี (Random Color Logic)

ในฟังก์ชัน `handlePress` ให้นิสิตสร้างตัวแปรเก็บรหัสสีสุ่ม เช่น:

```
const randomColor = '#' + Math.floor(Math.random()*16777215).toString(16);
```

แล้วนำไปอัปเดต State เพื่อให้สีขอบรูปเปลี่ยนทันทีที่กดปุ่ม

#### Step 4: สร้างปุ่มด้วย Pressable

สร้างปุ่มเพิ่มให้ครบ 3 ปุ่ม และปรับแต่ง `styles.button` ให้มีระยะห่างที่สวยงาม (แนะนำให้ใช้ `marginVertical` )

---

### 5. Test Table (not a checklist)

| Test Case | |
|---|---|
| รูปโปรไฟล์เป็นวงกลมและมีขอบตามเงื่อนไข | |
| จัดวางองค์ประกอบอยู่กึ่งกลางและสวยงาม (Flexbox) | |
| ตรรกะการสุ่มสีขอบรูปทำงานถูกต้องเมื่อกดปุ่ม | |
| การแสดงผล Alert ระบุชื่อแพลตฟอร์มถูกต้อง | |

---

**Tip:** ลองเพิ่มพร็อพเพอร์ตี้ `hitSlop` ใน `Pressable` เพื่อขยายพื้นที่กดให้กว้างขึ้นโดยไม่ต้องขยายขนาดปุ่ม

---

# Checklist Function 7.1

## C1. รูปโปรไฟล์เป็นวงกลมสมบูรณ์

- **วิธีดู:** ให้สังเกตที่รูปภาพ หากรูปเป็น **สี่เหลี่ยมมุมมน (Rounded Rectangle)** หรือ **วงรี** แสดงว่ายังไม่ผ่าน
- **เกณฑ์เทคนิค:** นิสิตต้องตั้งค่าใน `StyleSheet` ดังนี้:
  - `width` และ `height` ต้องเท่ากัน (เช่น 150)
  - `borderRadius` ต้องเท่ากับ **"ครึ่งหนึ่ง"** ของขนาด (เช่น 75)
  - ต้องมี `borderWidth` มากกว่า 0 จึงจะเห็นขอบสี

## C2. การจัดวาง Flexbox

- **วิธีดู:** องค์ประกอบทั้งหมด (รูป, ชื่อ, ปุ่ม) ต้องอยู่ **กึ่งกลางจอภาพพอดี** ทั้งแนวตั้งและแนวนอน แม้จะรันบนมือถือรุ่นที่จอต่างกัน (iPhone vs Android)
- **เกณฑ์เทคนิค:** ตรวจที่ `styles.container` ต้องมี 3 พร็อพเพอร์ตี้นี้:
  - `flex: 1` (เพื่อให้กินพื้นที่เต็มจอ)
  - `justifyContent: 'center'` (กลางแนวตั้ง)
  - `alignItems: 'center'` (กลางแนวนอน)

## C3. ตรรกะการสุ่มสีขอบรูป

- **วิธีดู:** เมื่อกดปุ่มใดก็ได้ (GitHub, Facebook ฯลฯ) **สีของเส้นรอบวงรูปโปรไฟล์ต้องเปลี่ยนทันที**
- **จุดที่ต้องระวัง:** สีต้องเปลี่ยนเป็น **สีใหม่แบบสุ่ม** ไม่ใช่เปลี่ยนแค่ 2-3 สีที่ตั้งค่าไว้
- **เกณฑ์เทคนิค:** ตรวจในฟังก์ชัน `handlePress`:
  - ต้องมีการเรียกใช้ฟังก์ชันสุ่มสี (เช่น `Math.random()`)
  - ต้องมีการเรียก `setBorderColor(newColor)` เพื่อ Update State

## C4. การตอบสนองด้วย Alert

- **วิธีดู:** เมื่อนิสิตกดปุ่ม ต้องมี **Pop-up (Alert)** เด้งขึ้นมา
- **เกณฑ์เทคนิค:** ข้อความใน Alert ต้องตรงกับปุ่มที่กด (เช่น กดปุ่ม GitHub ต้องขึ้นคำว่า "GitHub")
- **จุดตรวจสอบโค้ด:** นิสิตต้องส่ง Parameter เข้าไปในฟังก์ชัน `onPress={() => handlePress('GitHub')}` ได้ถูกต้อง ไม่ใช่เขียน Alert แยกกัน 3 ฟังก์ชัน

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

💡 **คำถาม :**

1. "ถ้าอยากให้รูปโปรไฟล์ใหญ่ขึ้นเป็น 200 ต้องแก้ `borderRadius` เป็นเท่าไหร่?" *(คำตอบ: 100)*
2. "ทำไมต้องใช้ `useState` ในการเก็บสี ทำไมใช้ตัวแปรธรรมดาไม่ได้?" *(คำตอบ: เพราะตัวแปรธรรมดาเปลี่ยนค่าแล้วหน้าจอไม่ Re-render ใหม่)*
3. "ถ้าอยากให้ปุ่มเรียงกันเป็นแนวนอน (ซ้ายไปขวา) ต้องไปแก้ที่ไหน?" *(คำตอบ: แก้ `flexDirection: 'row'` ใน View ที่หุ้มปุ่ม)*

Advanced Styling และการจัดการ Input โดยใช้เทคนิค Glassmorphism (เอฟเฟกต์กระจกฝ้า) ที่เป็นที่นิยมใน UI สมัยใหม่

---

# Function 7.2 - Dynamic Currency Converter (Glassmorphism UI)

---

## 1. วัตถุประสงค์

1. ฝึกการใช้ `TextInput` และการจัดการ Event `onChangeText`
2. ฝึกการทำสไตล์ขั้นสูงด้วยพร็อพเพอร์ตี้ `shadow`, `opacity` และการซ้อน Layer
3. เข้าใจการใช้งาน `KeyboardAvoidingView` เพื่อจัดการ UI เมื่อคีย์บอร์ดปรากฏขึ้น

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

## 2. โจทย์

พัฒนาแอปแปลงสกุลเงินบาท (THB) เป็นสกุลเงินอื่น โดยมีเงื่อนไขดังนี้:

1. **Glassmorphism Card**: ส่วนแสดงผลต้องดูเหมือนกระจกฝ้า (ใช้การผสมสีโปร่งแสงและเงา)
2. **Keyboard Management**: หน้าจอต้องไม่พังหรือถูกคีย์บอร์ดบังเมื่อผู้ใช้กำลังพิมพ์
3. **Real-time Calculation**: เมื่อตัวเลขในช่อง THB เปลี่ยน ค่าใน Card อื่นๆ (USD, JPY, EUR) ต้องคำนวณใหม่ทันที
4. **Validation**: ต้องกรอกได้เฉพาะตัวเลขเท่านั้น และต้องจัดการไม่ให้แอป Error เมื่อช่องกรอกว่างเปล่า

---

💡 **คำถาม**

1. "ทำไมเราต้องใช้ `parseFloat()` ก่อนนำค่าจาก TextInput มาคำนวณ?" *(คำตอบ: เพราะค่าจาก TextInput เป็น String ต้องแปลงเป็นตัวเลขก่อน)*
2. "พร็อพเพอร์ตี้ไหนที่ทำให้ Card ดูเหมือนกระจกฝ้า?" *(คำตอบ: การใช้สีที่มีค่า Alpha/Opacity และการใส่ Shadow)*
3. "`KeyboardAvoidingView` สำคัญยังไงใน Mobile App ที่ต่างจาก Web?" *(คำตอบ: เพราะในมือถือคีย์บอร์ดจะกินพื้นที่หน้าจอเกือบครึ่งหนึ่ง ถ้าไม่จัดการมันจะทับเนื้อหาที่ผู้ใช้กำลังพิมพ์)*

---

# Checklist Function 7.2

## C1. ตรรกะการคำนวณและการจัดการ Error

- **วิธีดู:** ลองพิมพ์ตัวเลขจำนวนมากลงในช่อง THB แล้วดูว่าค่าใน Card เปลี่ยนแปลงตามทันทีหรือไม่ (Real-time)
- **จุดชี้ขาด (The Crash Test):** ให้นักศึกษา **"ลบตัวเลขในช่อง Input ออกให้หมด"** * **ผ่าน:** แอปต้องยังรันอยู่ได้ และค่าใน Card แสดงเป็น `0` หรือ `0.00`

    - **ไม่ผ่าน:** แอปค้างหรือเด้งออก (Crash) เพราะโปรแกรมพยายามคำนวณเลขจากค่าว่าง (Empty String) หรือได้ผลลัพธ์เป็น `NaN`
- **เกณฑ์เทคนิค:** มีการใช้ `parseFloat()` หรือ `Number()` และมีการดัก `if (!thb)` ก่อนคำนวณ

## C2. ความสวยงามสไตล์ Glassmorphism

- **วิธีดู:** พื้นหลังของ Card ต้องมีความโปร่งแสง (Translucent) เห็นสีพื้นหลังของแอปจางๆ และดูมีมิติ
- **เกณฑ์เทคนิค:** ตรวจสอบใน `StyleSheet` ของนักศึกษา:
    - ต้องใช้สีแบบ `rgba(255, 255, 255, 0.2)` (ค่า 0.2 คือความใส)
    - **Android:** ต้องมี `elevation` มากกว่า 5
    - **iOS:** ต้องมี `shadowOpacity` และ `shadowRadius`

## C3. การใช้ Reusable Component

- **วิธีดู:** ตรวจดูในไฟล์โค้ด (Source Code) ของนักศึกษา
- **เกณฑ์เทคนิค:** นักศึกษาต้องไม่เขียนโค้ดแสดงผล Card ซ้ำๆ กัน 3 รอบ แต่ต้องสร้างคอมโพเนนต์แยกชื่อ `CurrencyCard` (หรือชื่ออื่น) และเรียกใช้งานโดยส่งผ่าน **Props** ( `label` และ `value` )
- **ทำไมต้องตรวจ:** เพื่อวัดว่านักศึกษาเข้าใจแนวคิดการเขียนโค้ดแบบแห้ง (DRY - Don't Repeat Yourself) หรือไม่

## C4. การจัดการ Keyboard & Input Type

- **วิธีดู:** เมื่อแตะที่ช่อง Input
    - คีย์บอร์ดที่เด้งขึ้นมาต้องเป็น **"แป้นตัวเลข"** เท่านั้น (ไม่มีตัวอักษร)
    - หน้าจอต้องมีการเลื่อนหลบ (Shift) ไม่ให้คีย์บอร์ดทับช่องที่กำลังพิมพ์
- **เกณฑ์เทคนิค:** * `TextInput` ต้องระบุ `keyboardType="numeric"` หรือ `decimal-pad`
    - มีการหุ้มเนื้อหาด้วย `KeyboardAvoidingView` และตั้งค่า `behavior` ให้เหมาะสมกับ OS

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

---

### 💡 คำถามทดสอบความเข้าใจ:

1. **"ถ้าต้องการเพิ่มสกุลเงิน 'KRW' (วอน) ต้องแก้โค้ดกี่บรรทัด?"**
    - *คำตอบที่ดี:* "แก้แค่ 2 บรรทัด คือเพิ่มค่าใน Object `rates` และเพิ่มบรรทัดเรียกใช้ `<CurrencyCard />` อีก 1 บรรทัด" (แสดงว่าวางโครงสร้าง Component มาดี)
2. **"ถ้าเปลี่ยนจาก `KeyboardAvoidingView` เป็น `View` ธรรมดา จะเกิดอะไรขึ้นกับแอป?"**
    - *คำตอบ:* "เวลาพิมพ์ คีย์บอร์ดจะเด้งขึ้นมาทับช่อง Input ทำให้มองไม่เห็นว่าพิมพ์อะไรลงไป"
3. **"ทำไมต้องใช้ `value={thb}` ใน TextInput ทั้งๆ ที่พิมพ์ลงไปก็โชว์อยู่แล้ว?"**
    - *คำตอบ:* "เพื่อให้เป็น Controlled Component ทำให้คุมค่าใน State และล้างค่าหรือเปลี่ยนค่าจากในโค้ดได้ง่าย"

การเชื่อมต่อ API จริง การจัดการ Navigation ที่ซับซ้อน และการรักษาสถานะข้อมูล (State) ข้ามหน้าจอ เพื่อเข้าใจ Data Flow อย่างแท้จริง

---

# Function 7.3 - The Deep-Link Library Explorer

---

## 1. วัตถุประสงค์

1. ฝึกการใช้ `React Navigation` (Stack Navigator) และการส่ง Params
2. ฝึกการดึงข้อมูลจาก API ภายนอกด้วย `Axios`
3. ฝึกการจัดการสถานะแบบเชื่อมโยง (State Synchronization) ระหว่างหน้าจอโดยไม่ใช้ Global State (เช่น Redux)

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

## 2. โจทย์

พัฒนาแอปพลิเคชันค้นหาหนังสือ โดยเชื่อมต่อกับ **Open Library API** มีเงื่อนไขดังนี้:

1. **Home Screen**: แสดงรายการหนังสือ (ชื่อเรื่อง และผู้แต่ง) โดยใช้ `FlatList`
2. **Detail Screen**: เมื่อกดที่หนังสือ ให้เปิดหน้ารายละเอียดเพื่อแสดงข้อมูลเพิ่มเติม
3. **Data Constraint (ยาก)**:
   - ในหน้า **Home** ห้ามโหลดข้อมูลผู้แต่ง (Author) ทั้งหมด ให้โหลดเฉพาะหัวข้อ
   - เมื่อกดเข้าหน้า **Detail** แอปต้องทำการดึงข้อมูล "รายละเอียดเนื้อหา (Description)" จาก API เพิ่มเติมโดยใช้ `work_id` ที่ส่งมาจากหน้าแรก
4. **Interaction (ยากมาก)**: ในหน้า Detail จะต้องมีปุ่ม **"Favorite ❤️"** เมื่อกดแล้ว เมื่อผู้ใช้กดย้อนกลับ (Back) มาที่หน้า Home รายการหนังสือชิ้นนั้นในหน้าแรกจะต้องมีข้อความว่า **"(Favorite)"** ปรากฏท้ายชื่อเรื่องทันที

---

## 3. Coding

ติดตั้ง Library ต่อไปนี้ก่อนเริ่ม:

```
npm install @react-navigation/native @react-navigation/native-stack axios
```

---

### 💡 คำถาม

1. "ทำไมเราไม่โหลด Description ของหนังสือทุกเล่มตั้งแต่หน้า Home?" *(คำตอบ: เพื่อประหยัด Data และทำให้แอปโหลดเร็วขึ้น (Performance optimization))*
2. "การส่งข้อมูลกลับจากหน้า Details มา Home ทำได้วิธีไหนบ้างนอกจาก Params?" *(คำตอบ: เช่นการใช้ Context API หรือ State Management อื่นๆ แต่ในที่นี้ Params ง่ายที่สุดสำหรับการส่งค่ากลับ (Callback pattern))*
3. "ทำไมต้องใส่ `keyExtractor` ใน `FlatList` ?" *(คำตอบ: เพื่อให้ React ระบุได้ว่า Item ไหนเปลี่ยนไป จะได้ Render เฉพาะส่วนนั้น (Virtualization))*

---

# Checklist Function 7.3

## C1. API Integration & Lazy Loading

- **วิธีดู:** 1. หน้าแรกต้องมีรายชื่อหนังสือขึ้นมา (ไม่ใช่ข้อมูลที่พิมพ์เอง)

  2. เมื่อกดเข้าหน้า Details ต้องเห็นคำว่า **"Loading..."** แวบหนึ่งก่อนที่เนื้อหาคำอธิบาย (Description) จะปรากฏ
- **จุดชี้ขาด:** หากหน้า Details แสดงข้อมูลทันทีโดยไม่มีการรอโหลด Description เพิ่มเติม แสดงว่าอาจจะแอบโหลดข้อมูลทั้งหมดมาตั้งแต่หน้าแรก (ซึ่งผิดเงื่อนไขเรื่อง Performance)
- **เกณฑ์เทคนิค:** ตรวจโค้ดในหน้า Details ว่ามีการใช้ `axios.get` หรือ `fetch` เพื่อดึงข้อมูลตาม `work_id` แยกต่างหากหรือไม่

## C2. Navigation & Params Passing

- **วิธีดู:** เมื่อกดที่รายการหนังสือในหน้า Home ข้อมูลที่ไปโผล่ในหน้า Details (เช่น ชื่อเรื่อง) ต้องตรงกับเล่มที่กด
- **เกณฑ์เทคนิค:** * หน้า Home ต้องเรียก `navigation.navigate('Details', { book: item })`
  - หน้า Details ต้องรับค่าผ่าน `const { book } = route.params`

## C3. State Synchronization

- **วิธีดู (The Favorite Test):**
  1. กดเข้าไปที่หนังสือเล่มที่ 1 -> กดปุ่ม "Mark as Favorite" -> กดย้อนกลับ
  2. **ผลที่ต้องเกิด:** ในหน้า Home หนังสือเล่มที่ 1 **ต้องมีหัวใจ ❤️ หรือคำว่า (Favorite)** ปรากฏขึ้นทันที
  3. **การทดสอบความแม่นยำ:** เล่มอื่นๆ ในรายการต้อง **ไม่มี** หัวใจขึ้นตาม (ต้องขึ้นเฉพาะเล่มที่กด)
- **เกณฑ์เทคนิค:** ต้องเขียน Logic ในหน้า Home เพื่อดักรับค่าที่ส่งกลับมา เช่น:

JavaScript

```
useEffect(() => {
  if (route.params?.favId) {
    // วนลูปเช็คใน State books แล้วอัปเดตเล่มที่มี id ตรงกับ favId
  }
}, [route.params?.favId]);
```

## C4. UI/UX Resilience

- **วิธีดู:** 1. ตอนโหลดข้อมูลต้องมี `ActivityIndicator` (วงล้อหมุน) แสดงผล ไม่ใช่ปล่อยหน้าจอขาวโพลน

  2. แอปต้องไม่ค้างหาก API ส่งข้อมูลกลับมาเป็นค่าว่าง (เช่น หนังสือบางเล่มไม่มี Description)
- **เกณฑ์เทคนิค:** มีการใช้ `.finally(() => setLoading(false))` เพื่อปิดตัวโหลดไม่ว่าจะสำเร็จหรือ Error

## C5. ตารางสรุปการทดสอบ

| จุดตรวจสอบ | ผ่าน (Pass) | ไม่ผ่าน (Fail) | ผลการทดสอบ |
|---|---|---|---|
| **Multi-step Fetching** | มีการดึงข้อมูลเพิ่มในหน้า Details (Lazy Load) | โหลดข้อมูลทุกอย่างมาตั้งแต่อยู่หน้าแรก | |
| **Navigation Flow** | ส่ง Object ข้ามหน้าและแสดงผลได้ถูกต้อง | ข้อมูลหน้า Details ไม่ตรงกับหน้า Home | |

| จุดตรวจสอบ | ผ่าน (Pass) | ไม่ผ่าน (Fail) | ผลการทดสอบ |
|---|---|---|---|
| **Data Sync (Back-pass)** | กดย้อนกลับมาแล้วสถานะ Favorite อัปเดตทันที | ต้องกด Refresh หรือปิดเปิดใหม่ ข้อมูลถึงจะเปลี่ยน | |
| **Loading States** | มีวงล้อโหลดข้อมูลให้ผู้เห็นทั้ง 2 หน้า | หน้าจอค้างนิ่งขณะรอเน็ต (ผู้ใช้ไม่รู้ว่าโหลดอยู่) | |

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

---

### 💡 คำถาม

1. **ถ้าเปลี่ยน `navigation.navigate('Home', ...)` เป็น `navigation.goBack()` ในหน้า Details ข้อมูล Favorite จะถูกส่งกลับไปได้ไหม?**
   - *คำตอบที่ถูกต้อง:* ไม่ได้ เพราะ `goBack()` ไม่สามารถแนบ Params กลับไปได้ ต้องใช้ `Maps` กลับไปยังหน้าเดิมพร้อมแนบ Params ใหม่
2. **ทำไมในหน้า Home เราต้องใช้ `useEffect` ดักจับ `route.params` ทั้งๆ ที่เพิ่งย้อนกลับมา?**
   - *คำตอบที่ถูกต้อง:* เพราะหน้า Home มันยังอยู่ใน Stack (ไม่ได้หายไปไหน) การที่ย้อนกลับมาพร้อม Params ใหม่ ทำให้ Component รับรู้การเปลี่ยนแปลงผ่าน `route.params` จึงต้องใช้ `useEffect` เพื่ออัปเดต State `books` ให้แสดงผลตามข้อมูลล่าสุด
3. **ถ้า API ของ Open Library ล่ม จะจัดการยังไงไม่ให้แอปขาวโพลน?**
   - *คำตอบที่ถูกต้อง:* ใช้ `try-catch` ครอบโค้ดส่วนดึงข้อมูล และถ้าเกิด Error ให้เซ็ต State error เพื่อแสดงข้อความแจ้งเตือนผู้ใช้แทนรายการหนังสือ

ความทนทานของแอป (Resilience) การจัดการสถานะ Network ที่ไม่แน่นอน และการทำ Cache

---

# Function 7.4 - The Real-time Multi-Source Dashboard

---

## 1. วัตถุประสงค์

1. ฝึกการจัดการ **Parallel API Requests** (การดึงข้อมูลจากหลายแหล่งพร้อมกัน)
2. ฝึกการใช้ **Axios Interceptors** เพื่อจัดการ Error และ Network ทั่วทั้งแอป
3. ฝึกการทำ **Offline-First Data Storage** ด้วย `AsyncStorage`

**หมายเหตุ** Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

## 2. โจทย์

พัฒนาหน้า Dashboard แสดงข้อมูลเศรษฐกิจ (Crypto Price) และสภาพอากาศ (Weather) โดยมีเงื่อนไขดังนี้:

1. **Parallel Fetching**: แอปต้องดึงข้อมูลจาก 2 APIs พร้อมกัน (เช่น CoinGecko และ OpenWeather) และต้องแสดง **Loading Spinner** ตัวเดียวที่หัวมุมจอ ซึ่งจะหายไปก็ต่อเมื่อ *ทั้งสอง* API โหลดเสร็จแล้วเท่านั้น
2. **Offline Mode** : หากผู้ใช้เปิดแอปขณะไม่มีอินเทอร์เน็ต แอปต้องไปดึงข้อมูล "การโหลดสำเร็จครั้งล่าสุด" จาก `AsyncStorage` มาแสดงแทน และขึ้นแถบเตือนสีส้มว่า **"Offline Mode: Showing Cached Data"**
3. **Partial Failure** : หาก API ตัวหนึ่งล่ม (เช่น Crypto ล่ม) แต่สภาพอากาศยังดึงได้ แอปต้องแสดงข้อมูลสภาพอากาศตามปกติ และแสดงข้อความ Error เฉพาะในส่วนของ Crypto (ห้ามล่มทั้งหน้าจอ)

---

## 3. Code Starter Template

นิสิตต้องติดตั้ง: `npm install axios @react-native-async-storage/async-storage`

---

### 💡 คำถาม

1. **ทำไมถึงใช้ `Promise.allSettled` แทนที่จะใช้ `Promise.all` ธรรมดา?**
   - *คำตอบที่ถูกต้อง:* "เพราะ `Promise.all` จะล้มเหลวทันทีถ้ามี API ตัวใดตัวหนึ่ง Error (All or Nothing) แต่ `allSettled` จะรอให้ทุกตัวทำงานเสร็จแม้จะมีบางตัวพัง ทำให้เราแยกจัดการ Partial Failure ได้
2. **ถ้าข้อมูลใน AsyncStorage กับข้อมูลบน Server ไม่ตรงกัน (Stale Data) ควรมีวิธีบอกผู้ใช้ยังไง?**
   - *คำตอบที่ถูกต้อง:* "ควรมี Timestamp บอกเวลาที่บันทึกข้อมูลล่าสุด (Last updated) เพื่อให้ผู้ใช้ตัดสินใจได้ว่าข้อมูลนั้นยังเชื่อถือได้หรือไม่
3. **Axios Timeout มีประโยชน์ยังไงในแอปมือถือ?**
   - *คำตอบที่ถูกต้อง:* "ป้องกันไม่ให้แอป 'รอเก้อ' เมื่อเน็ตช้ามากๆ ครับ ถ้าเกินเวลาที่กำหนด (เช่น 5 วินาที) ให้ยกเลิก Request และดึง Cache มาแสดงแทน เพื่อไม่ให้ผู้ใช้รู้สึกว่าแอปค้าง

---

# Checklist Function 7.4

## C1. Parallel Request Logic

- **วิธีดู:** สังเกตการโหลดข้อมูล
  - **ผ่าน:** ข้อมูลทั้ง 2 ส่วน (Crypto และ Weather) ต้องปรากฏขึ้นเกือบจะพร้อมกัน และตัวโหลด (Loading Spinner) ต้องแสดงผลเพียงอันเดียวที่คุมทั้งหน้าจอ
- **เกณฑ์เทคนิค:** ตรวจสอบโค้ดว่ามีการใช้ `Promise.all` หรือ `Promise.allSettled` หรือไม่
  - **ทำไมต้องตรวจ:** หากใช้ `await` ตัวที่ 1 จบแล้วค่อย `await` ตัวที่ 2 (Serial) ผู้ใช้งานที่มาดูข้อมูลจะรู้สึกว่าแอป "หน่วง" เกินจำเป็น

## C2. Resilience: Partial Failure Handling

- **วิธีดู (The Blackout Test):** ลองแก้ URL ของ API ตัวหนึ่งให้ผิด (เช่น เติมเลข 1 เข้าไปใน URL ของ OpenWeather)
  - **ผ่าน:** การ์ดของ Weather ต้องขึ้นคำว่า "Service Unavailable" หรือ "Error" แต่ **การ์ดของ Crypto ต้องยังทำงานและดึงข้อมูลได้ปกติ**
  - **ไม่ผ่าน:** หน้าแอปค้างหมุนไม่หยุด หรือขึ้นหน้าจอ Error สีแดง (Red Screen) แสดงว่าไม่ได้จัดการ Error แยกส่วนกัน

## C3. Offline Cache Capability

- **วิธีดู (The Flight Mode Test):** เป็นการทดสอบที่สำคัญที่สุด
  1. ให้ดึงข้อมูลให้สำเร็จ 1 ครั้ง (เพื่อให้มีข้อมูลลง Cache)
  2. สั่ง **ปิดเน็ต/เปิดโหมดเครื่องบิน** แล้วกดปุ่ม Sync หรือปิดแอปเปิดใหม่
  - **ผ่าน:** ข้อมูลเก่าที่เคยโหลดไว้ต้องยังปรากฏอยู่ (ไม่หายไป) และต้องมี **แถบแจ้งเตือนสีส้มหรือข้อความ** บอกชัดเจนว่านี่คือ "Offline Data" หรือ "Cached Data"
- **เกณฑ์เทคนิค:** ตรวจสอบว่ามีการเรียกใช้ `AsyncStorage.getItem` และ `setItem` ในจังหวะที่ดึงข้อมูลสำเร็จและล้มเหลวหรือไม่

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

---

### 💡 คำถามเช็คความเข้าใจระดับลึก

1. **ทำไมถึงลองให้ใช้ `timeout` ใน Axios? ถ้าไม่ใส่จะเกิดอะไรขึ้น?**
   - *คำตอบที่ถูกต้อง:* เพื่อให้แอปไม่รอเก้อ ถ้าเน็ตค้างหรือ Server ไม่ตอบสนองภายใน 5 วินาที แอปจะตัดจบและไปดึงข้อมูลใน Cache มาโชว์แทน เพื่อไม่ให้ User รอนานเกินไป
2. **ถ้าข้อมูลใน AsyncStorage เป็นข้อมูลของปีที่แล้ว ควรจัดการยังไง?**
   - *คำตอบที่ถูกต้อง:* ควรเก็บ Timestamp (เวลาที่บันทึก) ไว้ด้วยครับ และเขียน Logic เช็คว่าถ้าข้อมูลเก่าเกิน 1 วัน ให้แจ้งเตือนผู้ใช้ว่าข้อมูลหมดอายุ หรือไม่แสดงผลเลย
3. **ถ้า API 1 ตัวทำงานเร็วมาก (0.5 วินาที) แต่อีกตัวช้ามาก (4 วินาที) ผู้ใช้จะเห็นหน้าจอเป็นยังไง?**
   - *คำตอบที่ถูกต้อง:* "ถ้าใช้ `Promise.all` ผู้ใช้จะต้องรอ 4 วินาทีถึงจะเห็นทั้งสองอย่างพร้อมกัน ตัวโหลดจะหมุนจนกว่าตัวที่ช้าที่สุดจะเสร็จ

Components, Styling, Navigation, API, และ Storage สร้างระบบ Queue Sync เพื่อทำงานแบบ Offline

---

# Function 7.5 - The Offline-First Task Synchronizer

---

## 1. วัตถุประสงค์

1. ฝึกการสร้างระบบ **Background Synchronization** เบื้องต้น
2. ฝึกการจัดการข้อมูลแบบ **Queue (FIFO)** ใน `AsyncStorage`
3. ฝึกการใช้งาน **NetInfo API** เพื่อตรวจจับการเปลี่ยนแปลงสถานะเครือข่าย

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

## 2. โจทย์โดยละเอียด (Scenario)

ให้นิสิตพัฒนาแอปพลิเคชันบันทึกงาน (To-Do List) ที่เน้นความต่อเนื่องในการใช้งาน:

1. **Offline Insertion**: ผู้ใช้ต้องสามารถเพิ่ม Task ได้ตลอดเวลา แม้ไม่มีอินเทอร์เน็ต
2. **Local Persistence**: Task ที่เพิ่มตอน Offline ต้องถูกบันทึกในเครื่องและแสดงผลในลิสต์ทันที โดยมีสถานะกำกับว่า **"Pending ⏳ "**
3. **Auto-Sync Engine (ยากมาก)**: เมื่ออินเทอร์เน็ตกลับมาเชื่อมต่อ (Detected via NetInfo) แอปต้องทำการ "ไล่ส่ง" ข้อมูลที่ค้างอยู่ในเครื่อง (Queue) ขึ้นไปยัง Server ทีละรายการจนหมด
4. **Consistency**: เมื่อ Sync สำเร็จ สถานะต้องเปลี่ยนจาก "Pending ⏳ " เป็น **"Synced ✅ "** และห้ามส่งข้อมูลซ้ำซ้อน

---

## 3. Code Starter Template

นิสิตต้องติดตั้ง: `npm install @react-native-community/netinfo axios @react-native-async-storage/async-storage`

---

### 💡 คำถาม

1. **ทำไมต้องใช้ `Date.now().toString()` เป็น ID แทนที่จะรอ ID จาก Server?**
   - *คำตอบที่ถูกต้อง:* เพราะต้องสร้างรายการในเครื่องทันทีตอน Offline จึงต้องมี Unique ID ชั่วคราวเพื่อให้ React Native แสดงผลใน `FlatList` ได้โดยไม่ซ้ำกัน
2. **ถ้าเรามี Task ค้างอยู่ 100 รายการ แล้วเน็ตติดๆ ดับๆ ระบบ Sync จะพังไหม? จัดการยังไง?**
   - *คำตอบที่ถูกต้อง:* ต้องจัดการแบบ FIFO (First-In-First-Out) และใช้ `try-catch` ในลูป ถ้าตัวไหนส่งไม่ผ่านต้องหยุดหรือข้ามไปก่อน เพื่อไม่ให้ลำดับข้อมูลผิดพลาด
3. **ทำไมต้องมี `NetInfo.addEventListener` ? ใช้แค่เช็คเน็ตตอนกดปุ่ม Add ได้หรือไม่?**
   - *คำตอบที่ถูกต้อง:* "ไม่ได้ เพราะถ้าผู้ใช้เพิ่ม Task ตอน Offline ทิ้งไว้ แล้วเน็ตกลับมาทีหลังโดยที่เขาไม่ได้กด Add อีกรอบ แอปก็จะไม่รู้ตัว การใช้ Listener ทำให้แอปทำการ Sync ได้ทันทีที่เน็ตกลับมา

---

# Checklist Function 7.5

## C1. Offline Persistence & ID Generation

- **วิธีดู (The Airplane Test):** 1. ให้นักศึกษาเปิดโหมดเครื่องบิน (Offline)

  2. ลองเพิ่ม Task ใหม่ 2-3 รายการ
  - **ผ่าน:** รายการต้องปรากฏในลิสต์ทันที (แม้ไม่มีเน็ต) และสถานะต้องขึ้นว่า **"Pending ⏳ "**
  - **การทดสอบความชัวร์:** ให้ปิดแอป (Kill App) แล้วเปิดใหม่ในขณะที่ยัง Offline ข้อมูลที่เพิ่งเพิ่มต้องยังอยู่ครบ
- **เกณฑ์เทคนิค:** ตรวจสอบว่ามีการใช้ `AsyncStorage.setItem` ทุกครั้งที่กด Add และมีการสร้าง Unique ID ในฝั่ง Client (เช่น `Date.now()` )

## C2. Network Status Awareness

- **วิธีดู:** สังเกตแถบบาร์ด้านบน (Status Banner)
  - **ผ่าน:** เมื่อเปิด/ปิดเน็ต แถบสีต้องเปลี่ยนทันที (เช่น เขียว-Online / แดง-Offline) โดยไม่ต้องกด Refresh หน้าจอ
- **เกณฑ์เทคนิค:** มีการใช้งาน `NetInfo.addEventListener` ใน `useEffect` เพื่อดักฟังสถานะเครือข่ายตลอดเวลา

## C3. Automatic Background Sync

- **วิธีดู (The Magic Moment):** 1. ขณะที่มีรายการ "Pending" ค้างอยู่ ให้ทำการ **"เปิดเน็ต"**

  2. **ผ่าน:** โดยที่นักศึกษา **ห้ามกดปุ่มใดๆ** แอปต้องตรวจเจอเน็ตเอง และรายการ Pending ต้องเปลี่ยนสถานะเป็น **"Synced ✅ "** โดยอัตโนมัติทีละรายการ
- **เกณฑ์เทคนิค:** ตรวจสอบฟังก์ชัน `syncQueue` ว่ามีการวนลูป ( `map` หรือ `for...of` ) เพื่อส่งข้อมูลที่ค้างอยู่ใน AsyncStorage ขึ้น Server หรือไม่

## C4. Duplicate Prevention

- **วิธีดู:** หลังจาก Sync สำเร็จแล้ว ให้ลองปิดแอปแล้วเปิดใหม่อีกครั้ง
  - **ผ่าน:** รายการที่ Sync ไปแล้วต้องขึ้นสถานะ "Synced ✅" ตั้งแต่เริ่มเปิดแอป และต้องไม่เกิดการส่งข้อมูลซ้ำไปที่ Server อีกรอบ
- **เกณฑ์เทคนิค:** มีการอัปเดตสถานะใน AsyncStorage หลังจาก API ตอบกลับมาว่า Success

**<u>หมายเหตุ</u>** Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

### 💡 คำถามท้าทาย

1. **ถ้าเน็ตติดๆ ดับๆ ในขณะที่กำลัง Sync ข้อมูล 10 รายการ (ส่งสำเร็จไปแล้ว 5 เหลืออีก 5) แอปจะจัดการยังไง?**
   - *คำตอบที่คาดหวัง:* "ระบบจะหยุดทำงานและเก็บ 5 รายการที่เหลือไว้เป็น Pending เหมือนเดิม เมื่อเน็ตกลับมาอีกครั้งค่อยเริ่ม Sync จากตัวที่ 6 ต่อไป
2. **ทำไมเราถึงต้องใช้ `useEffect` คืนค่า (Cleanup Function) ให้กับ `NetInfo` ?**
   - *คำตอบที่คาดหวัง:* "เพื่อสั่งหยุดการดักฟัง (Unsubscribe) เมื่อผู้ใช้ออกจากหน้านี้ เพื่อไม่ให้เกิด Memory Leak หรือแอปทำงานหนักเกินไปโดยไม่จำเป็น
3. **ในแอปจริง ถ้า Server ตอบกลับมาว่า Error 400 (ข้อมูลผิดพลาด) จะยังเปลี่ยนสถานะเป็น Synced ไหม?**
   - *คำตอบที่คาดหวัง:* "ไม่ ต้องแสดงสถานะ Error และอาจจะต้องมีปุ่มให้ผู้ใช้แก้ไขข้อมูลก่อนจะพยายาม Sync ใหม่อีกครั้ง